

Accord — Protocol Specification v0.1

Status

Draft v0.1

Purpose

This document defines the first protocol model for Accord.

Its job is to turn the product requirements into a precise runtime grammar for:

- requesting access to user context,
- disclosing what context was used,
- proposing high-impact actions,
- capturing approval outcomes,
- revoking or expiring access,
- and recording all of the above in an auditable form.

This is a **v0.1 protocol draft**, intended to be simple, local-first, and implementation-friendly.

1. Design Goals

The protocol is designed around the following goals:

1.1 Explicitness

Important runtime decisions must be represented as structured objects, not hidden behind informal assistant behavior.

1.2 Inspectability

A developer or user should be able to inspect what happened and why.

1.3 Local-first simplicity

The protocol should work cleanly in a local environment before distributed or hosted concerns are introduced.

1.4 Fail-closed execution

If approval is required but missing, invalid, denied, or expired, the proposed action must not proceed.

1.5 Portability

The protocol should avoid framework lock-in and remain easy to map onto adjacent ecosystems.

1.6 Incremental extensibility

v0.1 should be minimal, but it should leave room for future fields and adapters without breaking core semantics.

2. Core Runtime Model

Accord v0.1 defines seven primary objects:

1. **ScopeRequest**
2. **GrantedScope**
3. **ContextManifest**
4. **ActionProposal**
5. **ApprovalReceipt**
6. **RevocationEvent**
7. **AuditRecord**

These objects represent the control-plane lifecycle of a context-aware, approval-sensitive AI interaction.

3. Shared Conventions

3.1 IDs

All major objects must carry a stable unique ID.

Recommended form for v0.1: - UUIDv7 or equivalent sortable unique identifier

3.2 Timestamps

All timestamps should be represented in ISO 8601 UTC format.

Example: - `2026-04-09T14:32:05Z`

3.3 Versioning

Every protocol object must carry: - `spec_version`

For v0.1: - `"0.1"`

3.4 Actor Types

The protocol recognizes the following actor roles in v0.1: - `user` - `application` - `agent` - `system`

3.5 Risk Levels

v0.1 uses a simple risk model: - `low` - `medium` - `high` - `critical`

This is intentionally minimal and may evolve later.

3.6 Decision Outcomes

Approval or policy outcomes in v0.1 may be: - `approved` - `denied` - `expired` - `revoked` - `pending`

3.7 Enforcement Modes

Policy enforcement in v0.1 may yield: - `allow` - `require_approval` - `deny`

4. ScopeRequest

4.1 Purpose

A `ScopeRequest` is created when an application or agent requests access to one or more categories of user context.

It answers: - who is asking, - what they want access to, - why they want it, - and for how long.

4.2 Required Fields

```
{
  "id": "string",
  "spec_version": "0.1",
  "requester": {
    "actor_type": "application|agent",
    "id": "string",
    "name": "string"
  },
  "context_categories": ["string"],
  "purpose": "string",
  "requested_ttl_seconds": 3600,
  "created_at": "ISO-8601 timestamp"
}
```

4.3 Optional Fields

```
{
  "sensitivity_hint": "low|medium|high|critical",
  "justification": "string",
  "metadata": {}
}
```

4.4 Semantics

- A `ScopeRequest` does not itself grant access.
 - It is an input to policy evaluation and/or user approval.
 - The request may result in a `GrantedScope`, a denial, or a deferred/pending state.
-

5. GrantedScope

5.1 Purpose

A `GrantedScope` is the materialized result of a successful scope decision.

It describes the context access that is currently valid.

5.2 Required Fields

```
{
  "id": "string",
  "spec_version": "0.1",
  "scope_request_id": "string",
  "grantee": {
    "actor_type": "application|agent",
    "id": "string",
    "name": "string"
  },
  "granted_categories": ["string"],
  "purpose": "string",
  "enforcement_mode": "allow|require_approval",
  "issued_at": "ISO-8601 timestamp",
  "expires_at": "ISO-8601 timestamp",
  "status": "active|expired|revoked"
}
```

5.3 Optional Fields

```
{
  "constraints": {
    "max_uses": 10,
    "allowed_targets": ["string"],
    "notes": "string"
  },
  "metadata": {}
}
```

5.4 Semantics

- A `GrantedScope` is the authority boundary for context use.
- A scope can be active, expired, or revoked.
- A scope may still require approval for certain actions even when granted.

6. ContextManifest

6.1 Purpose

A `ContextManifest` declares what context was actually used in relation to an action proposal.

This object is central to Accord's value.

It answers: - what categories of context influenced the action, - what concrete entries or sources were involved, - and what their relationship to the action was.

6.2 Required Fields

```
{
  "id": "string",
  "spec_version": "0.1",
  "granted_scope_id": "string",
  "used_categories": ["string"],
  "used_entries": [
    {
      "entry_id": "string",
      "category": "string",
      "source": "string",
      "summary": "string"
    }
  ],
}
```

```
    "generated_at": "ISO-8601 timestamp"
}
```

6.3 Optional Fields

```
{
  "provenance": [
    {
      "entry_id": "string",
      "origin_type": "memory|preference|rule|manual_input|system",
      "origin_ref": "string"
    }
  ],
  "notes": "string",
  "metadata": {}
}
```

6.4 Semantics

- A `ContextManifest` is not the raw user data store.
- It is a disclosure object describing what was actually used.
- The summaries in `used_entries` should be human-legible wherever possible.
- The manifest should be narrow enough that a user can understand the action context without reading the full underlying data store.

7. ActionProposal

7.1 Purpose

An `ActionProposal` represents an external or meaningful side effect the system wants to perform.

Examples: - write a file - commit code - push to remote - open a PR - publish content - submit a form - send a message

7.2 Required Fields

```
{
  "id": "string",
  "spec_version": "0.1",
  "requester": {
    "actor_type": "application|agent",
    "id": "string",
    "name": "string"
  }
}
```

```

    },
    "action_type": "string",
    "target": {
      "type": "string",
      "ref": "string"
    },
    },
    "summary": "string",
    "risk_level": "low|medium|high|critical",
    "context_manifest_id": "string",
    "created_at": "ISO-8601 timestamp",
    "status": "pending"
  }
}

```

7.3 Optional Fields

```

{
  "execution_payload": {},
  "human_readable_diff": "string",
  "reason": "string",
  "policy_hint": "allow|require_approval|deny",
  "expires_at": "ISO-8601 timestamp",
  "metadata": {}
}

```

7.4 Semantics

- An `ActionProposal` is created before execution.
- It must be evaluated by policy.
- If the policy outcome is `require_approval`, execution must wait for an `ApprovalReceipt` with an `approved` outcome.
- If the action expires before approval, it must fail closed.

8. ApprovalReceipt

8.1 Purpose

An `ApprovalReceipt` is the machine-checkable outcome of a review or policy-controlled approval step.

It is the object an executor checks before proceeding.

8.2 Required Fields

```
{
  "id": "string",
  "spec_version": "0.1",
  "action_proposal_id": "string",
  "decision": "approved|denied|expired|revoked|pending",
  "decider": {
    "actor_type": "user|system",
    "id": "string",
    "name": "string"
  },
  "decided_at": "ISO-8601 timestamp"
}
```

8.3 Optional Fields

```
{
  "reason": "string",
  "valid_until": "ISO-8601 timestamp",
  "conditions": {
    "single_use": true,
    "target_lock": "string",
    "notes": "string"
  },
  "metadata": {}
}
```

8.4 Semantics

- A receipt with `decision = approved` authorizes execution only within any attached conditions.
- A receipt may be single-use.
- A receipt may expire.
- A denied, expired, or revoked receipt must not permit execution.
- A system-generated receipt may exist for low-risk auto-approved cases if policy allows it.

9. RevocationEvent

9.1 Purpose

A `RevocationEvent` represents a withdrawal of previously valid scope or approval authority.

9.2 Required Fields

```
{
  "id": "string",
  "spec_version": "0.1",
  "revocation_type": "scope|approval",
  "target_id": "string",
  "revoked_by": {
    "actor_type": "user|system",
    "id": "string",
    "name": "string"
  },
  "revoked_at": "ISO-8601 timestamp"
}
```

9.3 Optional Fields

```
{
  "reason": "string",
  "metadata": {}
}
```

9.4 Semantics

- Revocation invalidates future use of the targeted scope or receipt.
- Revocation does not erase history.
- Revocation should be represented in the audit trail.

10. AuditRecord

10.1 Purpose

An `AuditRecord` captures a durable event in the Accord control-plane lifecycle.

10.2 Required Fields

```
{
  "id": "string",
  "spec_version": "0.1",
  "event_type": "string",
  "object_type": "ScopeRequest|GrantedScope|ContextManifest|ActionProposal|
ApprovalReceipt|RevocationEvent",
  "object_id": "string",

```

```
"recorded_at": "ISO-8601 timestamp"
}
```

10.3 Optional Fields

```
{
  "actor": {
    "actor_type": "user|application|agent|system",
    "id": "string",
    "name": "string"
  },
  "summary": "string",
  "metadata": {}
}
```

10.4 Semantics

- Every significant state transition should generate an audit record.
- Audit records should be append-only in the reference implementation.
- Audit history should be inspectable in the local console.

11. Policy Evaluation Model

11.1 Purpose

The policy engine determines whether a request or action is: - allowed, - approval-required, - or denied.

11.2 Inputs

At minimum, policy evaluation may consider: - requester identity - context categories - sensitivity hints - purpose - action type - target type - risk level - granted scope status - expiration status

11.3 Outputs

Policy evaluation in v0.1 must return one of: - `allow` - `require_approval` - `deny`

11.4 Example Rules

Examples of local rules in v0.1: - low-risk read-only actions may be auto-allowed - file writes may require approval - pushes to remote may require approval - expired scopes always deny - revoked scopes always deny - critical-risk actions default to approval or deny

11.5 Semantics

- Policy is authoritative for execution gating.
 - An executor must not bypass policy outcome.
 - Policy results should be reflected in audit history.
-

12. Execution Contract

12.1 Principle

Execution is downstream of Accord, not part of Accord itself.

12.2 Required Check

Before performing a side effect, the executor must verify: 1. the referenced `GrantedScope` exists and is active, 2. the `ContextManifest` is valid, 3. the `ActionProposal` exists and is not expired, 4. the policy outcome is not `deny`, 5. and if approval is required, a valid `ApprovalReceipt` with `approved` decision exists.

12.3 Failure Behavior

If any required object is missing, invalid, expired, revoked, or denied: - execution must fail closed.

13. Minimal End-to-End Flow

Step 1 — Scope Request

The application creates a `ScopeRequest`.

Step 2 — Scope Grant

The server evaluates policy and creates a `GrantedScope` if allowed.

Step 3 — Context Use

The application retrieves permitted context under the granted scope.

Step 4 — Context Manifest

The application creates a `ContextManifest` describing what it actually used.

Step 5 — Action Proposal

The application creates an `ActionProposal`.

Step 6 — Policy Evaluation

The server evaluates the proposal and returns `allow`, `require_approval`, or `deny`.

Step 7 — Approval

If required, the user reviews the proposal and the system issues an `ApprovalReceipt`.

Step 8 — Execution

The executor checks the receipt and performs the action only if conditions are satisfied.

Step 9 — Audit Trail

Every step produces `AuditRecord` entries.

14. Example JSON Fragments

14.1 Example ScopeRequest

```
{
  "id": "sr_01",
  "spec_version": "0.1",
  "requester": {
    "actor_type": "agent",
    "id": "agent.local.coder",
    "name": "Local Coding Agent"
  },
  "context_categories": ["project_preferences", "git_rules"],
  "purpose": "prepare file edit and git commit proposal",
  "requested_ttl_seconds": 1800,
  "created_at": "2026-04-09T14:32:05Z"
}
```

14.2 Example ContextManifest

```
{
  "id": "cm_01",
  "spec_version": "0.1",
  "granted_scope_id": "gs_01",
  "used_categories": ["project_preferences", "git_rules"],
  "used_entries": [
    {
      "entry_id": "pref_12",

```

```

    "category": "project_preferences",
    "source": "local_vault",
    "summary": "Prefer small commits and descriptive commit messages"
  },
  {
    "entry_id": "rule_03",
    "category": "git_rules",
    "source": "local_vault",
    "summary": "Never push directly to main"
  }
],
"generated_at": "2026-04-09T14:34:10Z"
}

```

14.3 Example ActionProposal

```

{
  "id": "ap_01",
  "spec_version": "0.1",
  "requester": {
    "actor_type": "agent",
    "id": "agent.local.coder",
    "name": "Local Coding Agent"
  },
  "action_type": "git.commit",
  "target": {
    "type": "repository",
    "ref": "repo://local/accord"
  },
  "summary": "Create a commit with the current staged file changes",
  "risk_level": "medium",
  "context_manifest_id": "cm_01",
  "created_at": "2026-04-09T14:35:00Z",
  "status": "pending"
}

```

14.4 Example ApprovalReceipt

```

{
  "id": "ar_01",
  "spec_version": "0.1",
  "action_proposal_id": "ap_01",
  "decision": "approved",
  "decider": {
    "actor_type": "user",

```

```
    "id": "user.local",
    "name": "Local User"
  },
  "decided_at": "2026-04-09T14:36:21Z",
  "conditions": {
    "single_use": true,
    "target_lock": "repo://local/accord"
  }
}
```

15. Deferred for Future Versions

The following are explicitly deferred beyond v0.1:

- cryptographic signing of receipts
- distributed or multi-device synchronization
- multi-user delegation
- organization policy inheritance
- richer risk scoring models
- privacy-preserving redaction layers
- field-level consent semantics
- deep identity integration
- formal capability grants beyond local reference usage

These may come later, but they should not complicate the first version.

16. Open Questions

The following questions remain for the architecture and implementation documents:

1. What persistence model best stores these objects locally?
 2. Should the policy engine use static rules, JSON config, or a lightweight DSL in v1?
 3. How should execution adapters verify receipt validity in practice?
 4. What exact minimal API endpoints should expose these objects?
 5. How should object relationships be indexed for fast console inspection?
 6. What should the SDK ergonomics look like around common flows?
-

17. Closing Statement

Accord v0.1 defines a minimal but rigorous runtime grammar for approval-aware personalization.

It treats context use and action approval as explicit control-plane objects rather than invisible application logic.

That is the foundation the SDK, server, policy engine, console, and adapters will build on.